

Código Intermedio

Conceptos Básicos



1. ¿Qué es el Código Intermedio?

2

Simplemente, se llama **Código Intermedio** a un código (programa) escrito en un lenguaje especial, llamado **Lenguaje Intermedio**. A éste término se lo abrevia **IL**, del inglés **Intermediate Language**.

Por ejemplo:

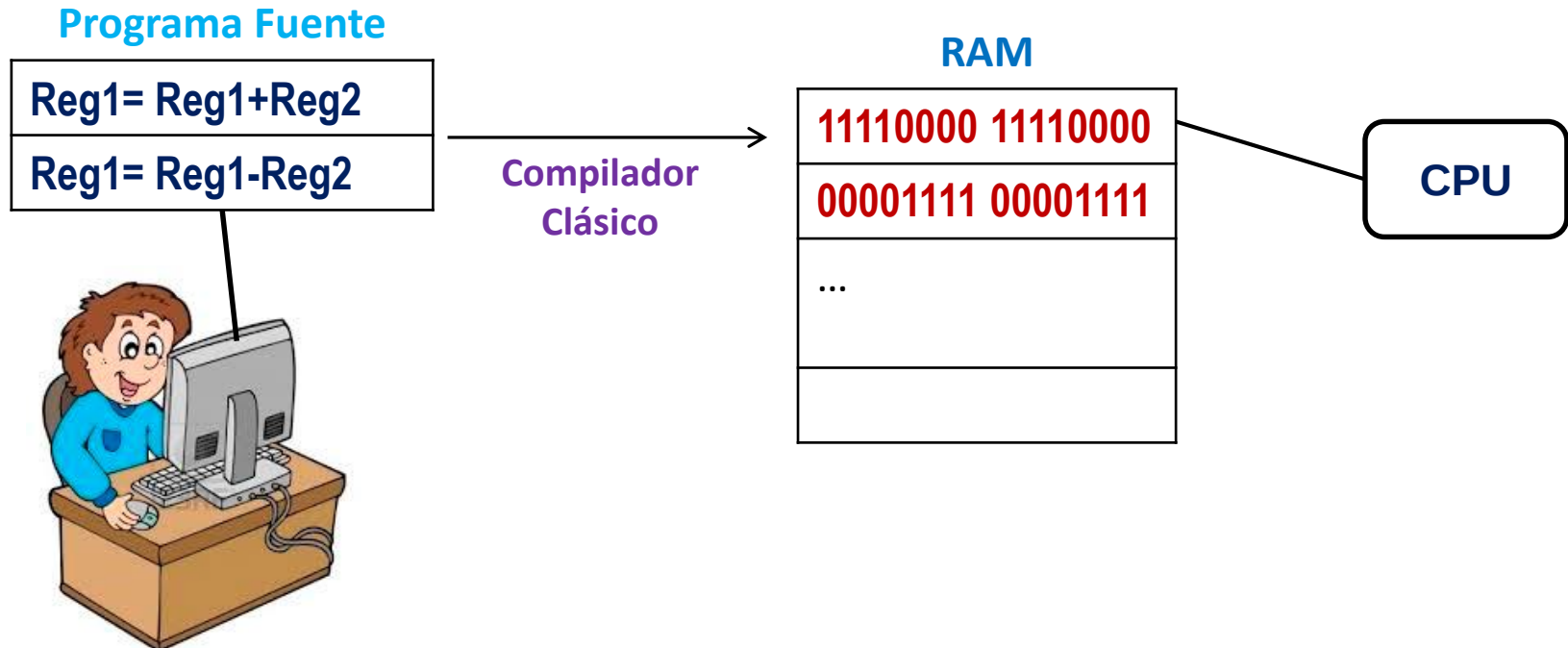
```
read(x)
read(y)
t1=(x > y)
if (t1=0) => goto E1
...
```

Este programa está escrito en un **IL**. Por tanto, se dice que este programa es un **Código Intermedio**.

Los términos “Código Intermedio” y “Lenguaje Intermedio”, coloquialmente (vulgarmente) son sinónimos. Es decir, informalmente, es común decirle “Código Intermedio” a las instrucciones de un Lenguaje Intermedio.

1.1 El Lenguaje Máquina

Como se sabe, cada familia de microprocesadores (**CPU's**) entiende un lenguaje binario llamado “Código o Lenguaje Máquina”. Así, cada **instrucción** del Lenguaje Máquina es (o se codifica como) un **número binario**.



1.1 El Lenguaje Máquina

Por tanto, es prácticamente improbable que las instrucciones de una familia de CPU's se codifique con los mismos números binarios que usan las instrucciones de otra familia de CPU's. Y peor aún, en una familia de CPU's pueden haber instrucciones que no hay en la otra familia y viceversa.

Ejemplificando, imaginemos que tenemos un programa con dos instrucciones:

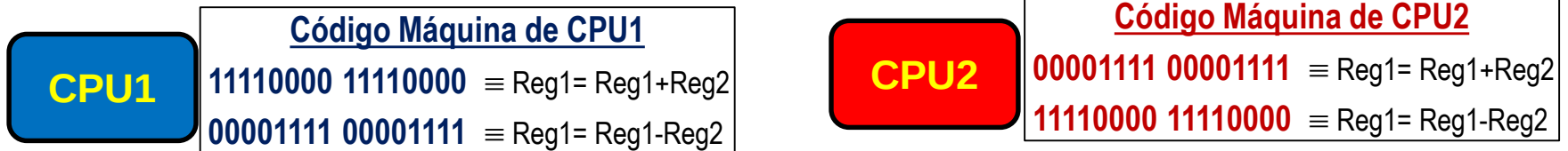
Reg1=Reg1+Reg2 ;

Reg1=Reg1-Reg2 ;

y dos familias de microprocesadores **CPU1** y **CPU2** cuyas instrucciones de **16 bits***, por ejemplo, se codifican así:

*De 16 bits significa que cada instrucción se representa, a lo sumo, por un número binario de 16 bits = 2 bytes.

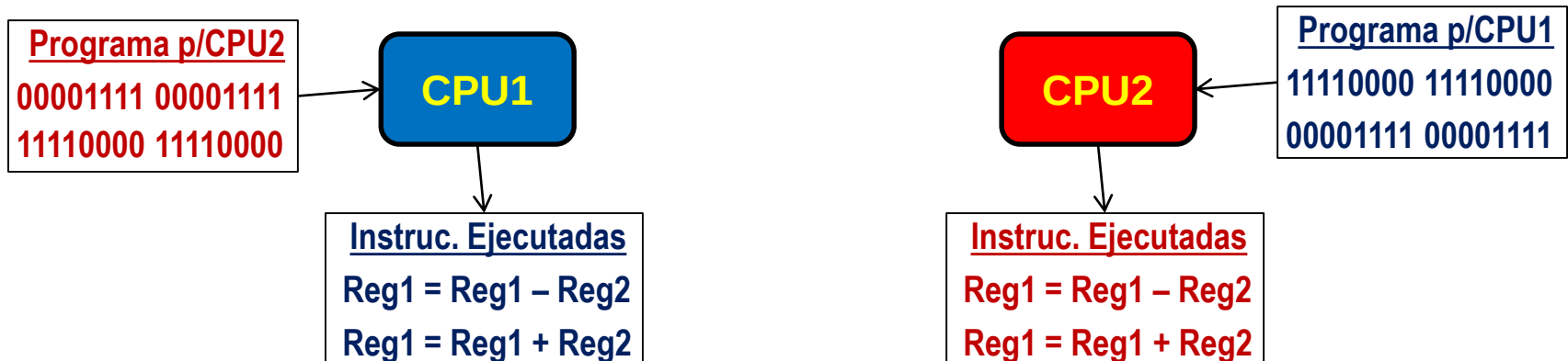
1.1 El Lenguaje Máquina



Ahora, preguntémonos :

¿Qué pasaría si, erróneamente, las instrucciones de nuestro programa codificado en binario para la CPU2, lo corremos en la CPU1 y viceversa?

Respuesta.- La **CPU1** interpretará los códigos binarios y hará las instrucciones que corresponden a su codificación, obteniendo “Reg1=Reg1-Reg2; Reg1=Reg1+Reg2”; es decir, **al revés** de lo que esperamos (“Reg1=Reg1+Reg2; Reg1=Reg1-Reg2”). Lo mismo pasaría si en la **CPU2**, corremos las instrucciones de la **CPU1**.



1.2 El Lenguaje Ensamblador

Entonces, si queremos escribir programas directamente en el Lenguaje Máquina para una **CPU1** tenemos que conocer/deducir todas y c/u de los **números binarios** que representan sus **instrucciones**, lo cual nos llevaría mucho tiempo de estudio e investigación, sin contar con lo moroso que sería el desarrollo de nuestros programas.

Ahora, supongamos que de pronto aparece una **CPU2** y estamos interesados en desarrollar software (escrito en Lenguaje Máquina) para él. Nuevamente, tenemos que leer, estudiar e investigar su **codificación binaria** antes de escribir nuestro primer programa.

Para ahorrarnos la tediosa tarea de escribir números binarios directamente, se crea el **Ensamblador*** o **assembler** el cual es un **software** que **traduce** uno a uno los mnemónicos (**instrucciones en modo texto**) a su respectiva **codificación binaria**.

*Por un abuso lingüístico, ampliamente aceptado, se le suele llamar “Ensamblador” al **Lenguaje** que manipula el software y no al **software** en sí. Así, es común escuchar: “Este es un programa escrito en Ensamblador”; aunque lo correcto sería decir: “Este es un programa escrito en el **lenguaje del Ensamblador**”.

1.2 El Lenguaje Ensamblador

Desafortunadamente, cada familia de microprocesadores crea **su propio** juego de mnemónicos (instrucciones escritas como un texto) y por tanto un programa assembler (ASM) escrito para un modelo de CPU, seguramente será diferente para otro modelo de CPU.

Por ejemplo, si queremos escribir el programa “Reg1=Reg1+Reg2; Reg1=Reg1-Reg2”, en los assembler’s de la **Intel 8086** y de la **IBM370**, debemos escribir sendas versiones del mismo.

ASM p/Intel 8086

ADD AX, BX //Mnemónico de Reg1=Reg1+Reg2

SUB AX, BX //Mnemónico de Reg1=Reg1- Reg2

ASM p/IBM370

AR R1, R2 //Mnemónico de Reg1=Reg1+Reg2

SR R1, R2 //Mnemónico de Reg1=Reg1- Reg2

Resumiendo las secciones 1.1 y 1.2, se puede decir que el Lenguaje Máquina y el Ensamblador son lenguajes específicos para **una** Computadora en particular. Es decir, si codificamos nuestros programas en Lenguaje Máquina o en Assembler, éstos sólo correrán en las computadoras que son capaces de soportarlos (entenderlos).

2. El Lenguaje Intermedio

Un **Lenguaje Intermedio** o **Intermediate Language (IL)** es un lenguaje de programación, cuyas instrucciones son del **mismo nivel del Lenguaje Máquina**, pero es **independiente** del Lenguaje Máquina de una computadora en particular.

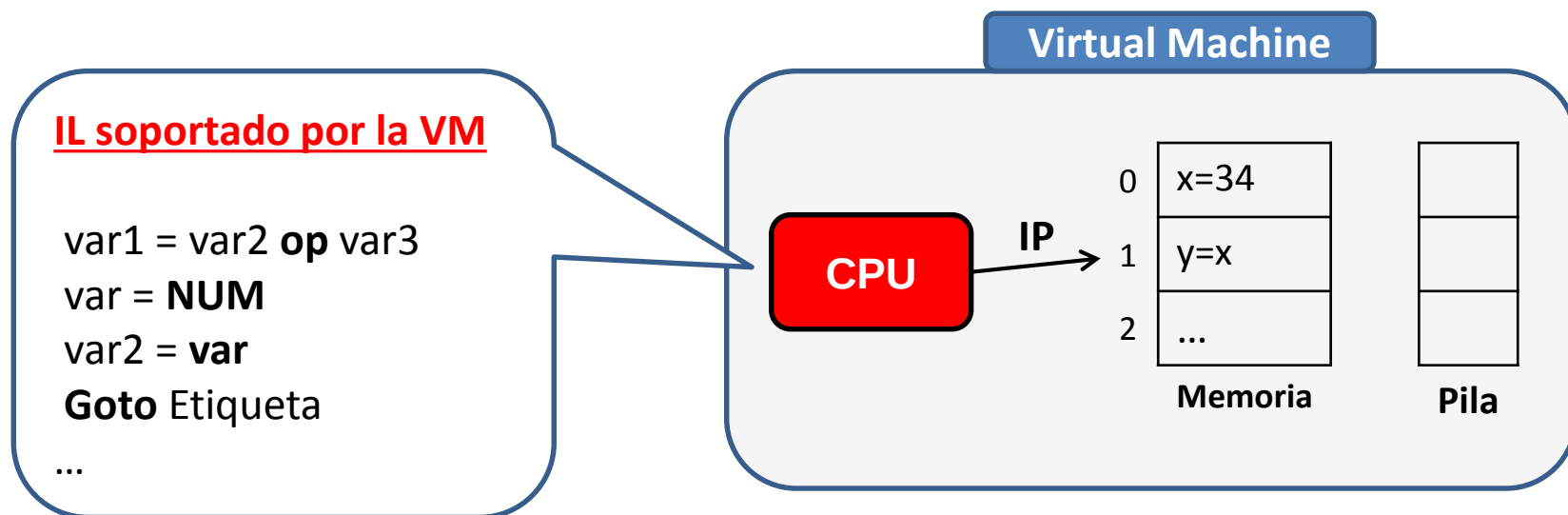
Decir que un **IL** tiene el “*mismo nivel del lenguaje máquina*”, es equivalente a afirmar que sus **instrucciones son muy básicas**. En otras palabras, si hiciéramos mnemónicos de un **Código Intermedio** (programa escrito en un **IL**) escribiríamos una colección de **instrucciones parecidas al assembler**.

Señalar que un **IL** es “*independiente del Lenguaje Máquina de una computadora en particular*”, es equivalente a decir que **ninguna computadora** conoce o entiende a un **IL**.

2. El Lenguaje Intermedio

¿Pero, si un programa escrito en un **Lenguaje Intermedio (IL)** no puede ser ejecutado (run) por ninguna computadora, **quién lo ejecutará?**

Respuesta.— Un **IL** es un **lenguaje inventado** por alguna persona, que imagina que su Lenguaje Intermedio será soportado (entendido) por una **Máquina** (computadora) **Abstracta** (imaginaria). A esta “computadora imaginaria”, formalmente, se le llama **Máquina Virtual** o **Virtual Machine** (cuya abreviatura es **VM**).

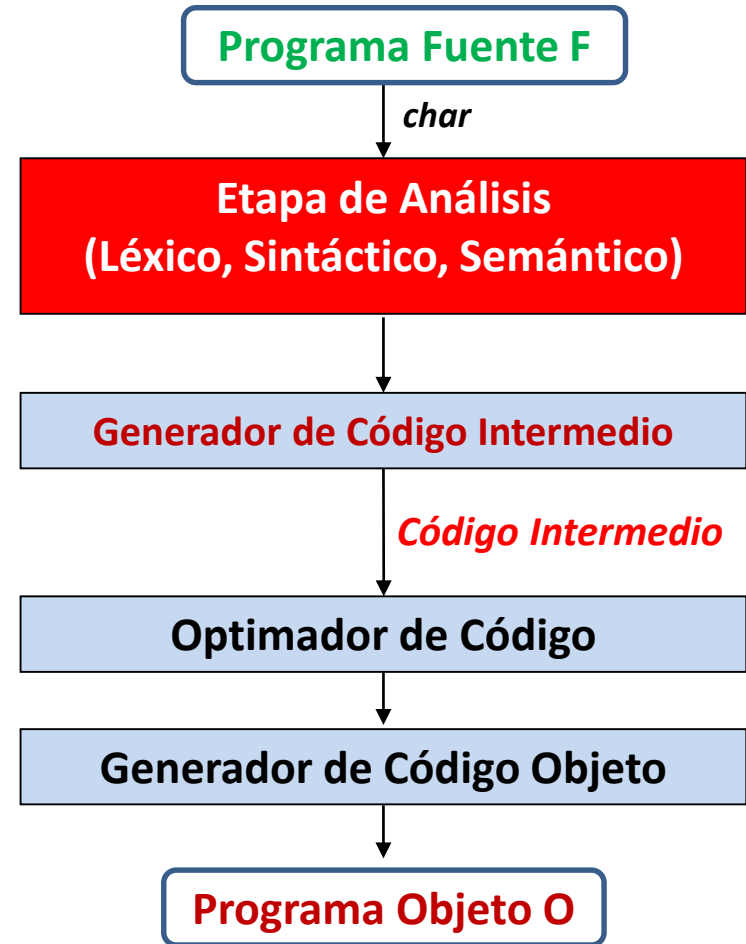


En esta gráfica, por ejemplo, se muestra una **VM** para el **IL** llamado **Código-3**.

2. El Lenguaje Intermedio

10

Puesto que los **Lenguajes Intermedios (IL)**, son muy cercanos al assembler (ASM), inicialmente se crearon con la intención de ayudar a los Compiladores Clásicos a optimar el **código objeto absoluto***. Es decir, a reducir el código absoluto, de modo que éste sea más rápido.

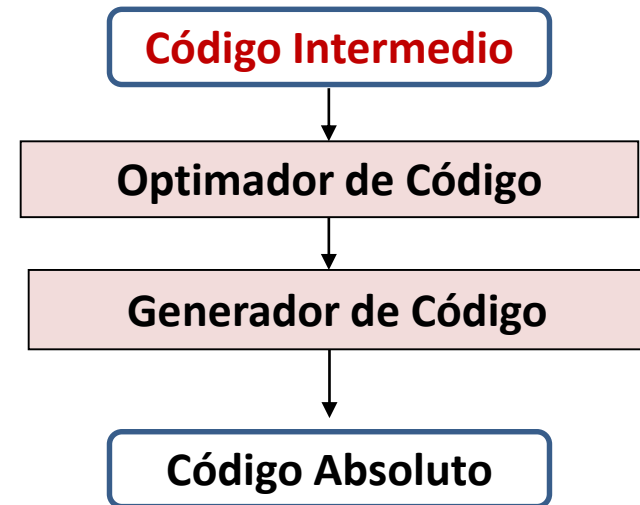
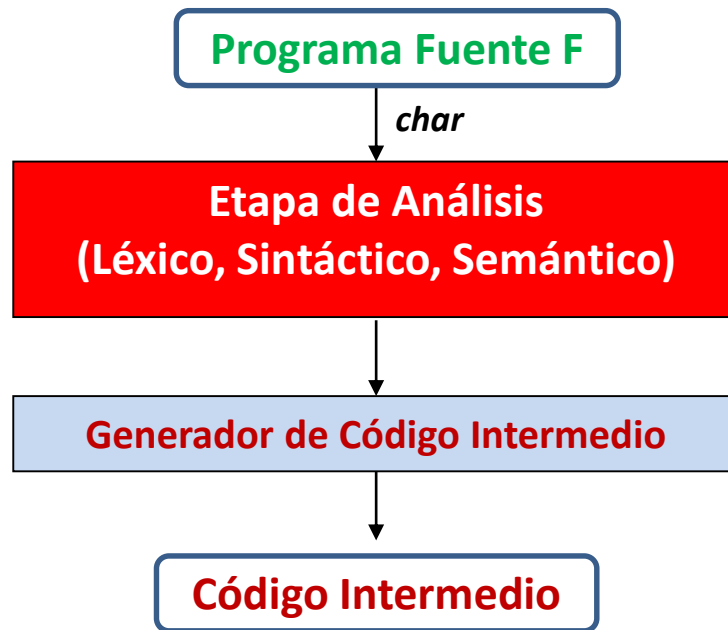


*Código Objeto Absoluto = Código escrito en Lenguaje Máquina o ASM.

2.1 Front-End y Back-End de un Compilador.

11

Actualmente, muchos Compiladores usan el Lenguaje Intermedio para generar diferentes Códigos Absolutos. El Compilador en sí, solo traduce el **Programa Fuente F** a un Lenguaje Intermedio (IL) y luego éste **Código Intermedio** se traduce al **Código Absoluto** que se desee.

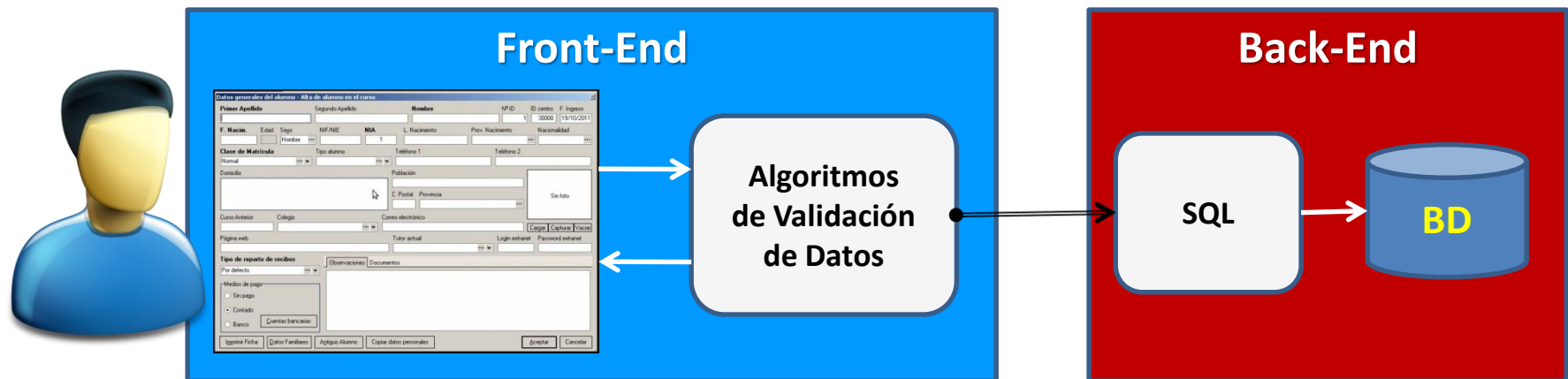


2.1. Front-End y Back-End de un Compilador.

Front-End y **Back-End** (traducibles al español como **interfaz** y **motor**, respectivamente) son términos que se relacionan con el principio y el final de un **proceso**.

En la **Ingeniería de Software** el **Front-End** es la parte del software que interactúa con el o los usuarios y el **Back-End** es la parte que procesa la entrada desde el **Front-End**. Lo que se quiere es que el **Front-End** sea el responsable de recolectar los datos de entrada del usuario, que pueden ser de muchas y variadas formas, y que los **transforme adecuadamente** a las especificaciones que demanda el **Back-End** para poder procesarlos.

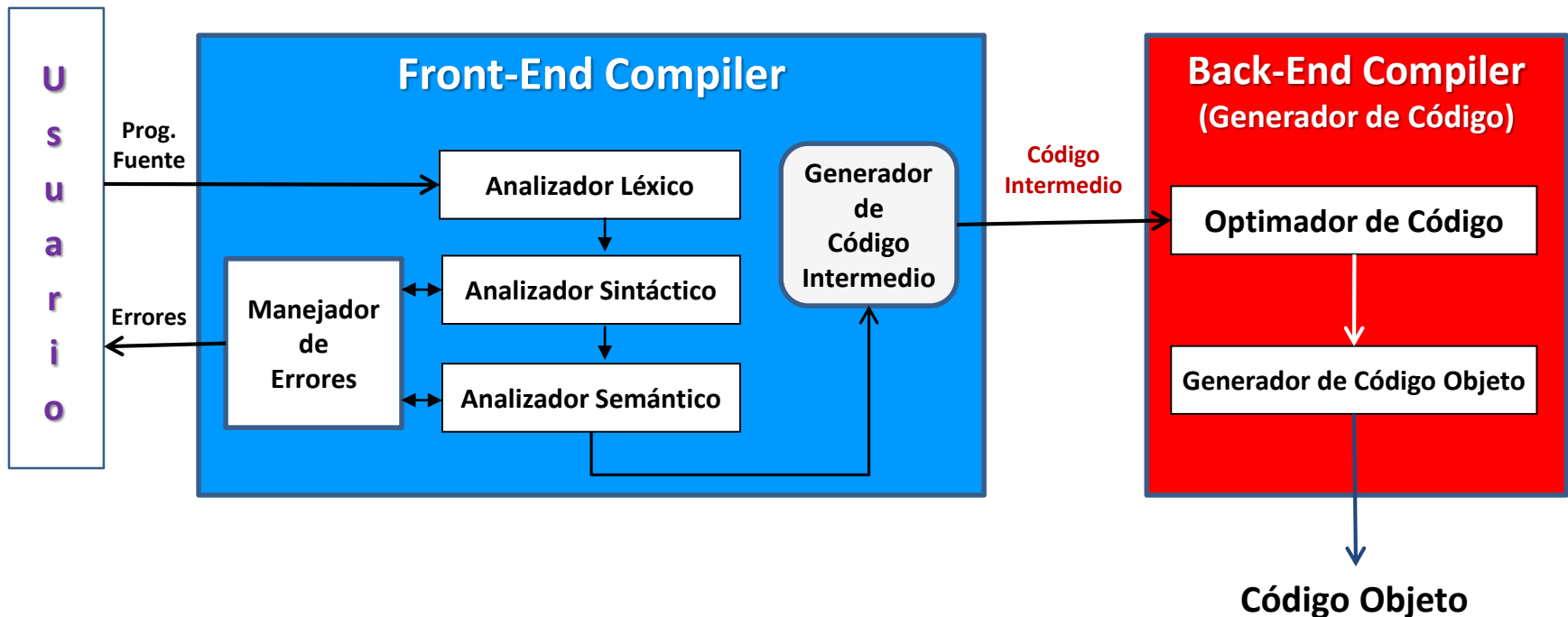
Por ejemplo, para darnos una idea, supongamos que tenemos un pequeño software que nos permite inscribir alumnos a un curso en particular. El **Front-End** de este programa abarcará la Interfaz del usuario y la validación de los datos introducidos en ella, mientras que el **Back-End** será la inserción de estos datos (ya validados) a la Base de Datos (BD).



2.1. Front-End y Back-End de un Compilador.

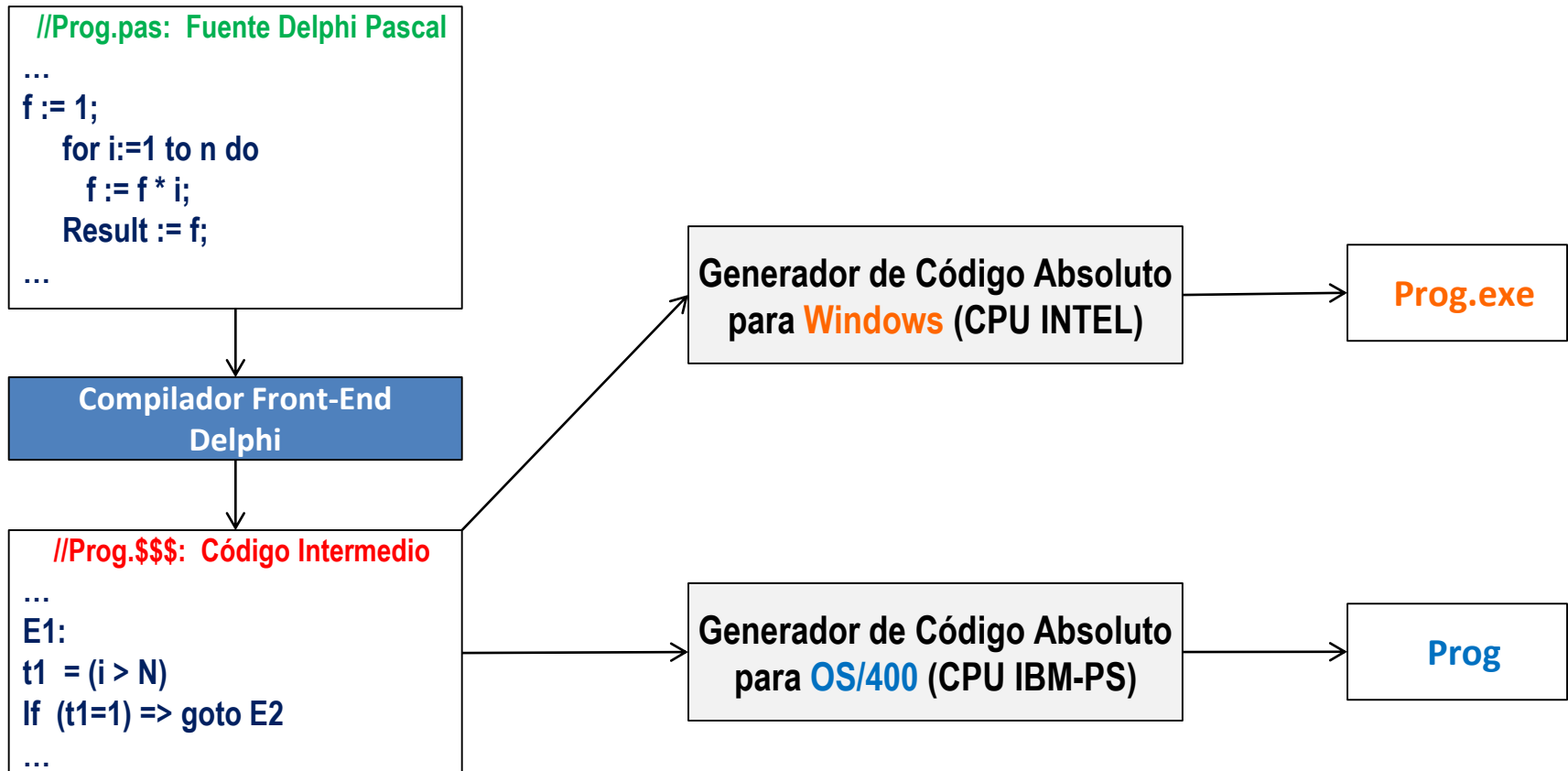
Si usamos el gráfico típico del **Front-End** y **Back-End**, veremos al Compilador como el mostrado en la figura inferior. Por “**Usuario**” estamos considerando al **Programador** o al **IDE**.

La tarea del **Front-End** empieza con el Analizador Léxico y termina con el **Código Intermedio** (programa escrito en un **IL**) generado. La labor del **Back-End** es tomar al **Código Intermedio** y generar el **Código Objeto** que se desee



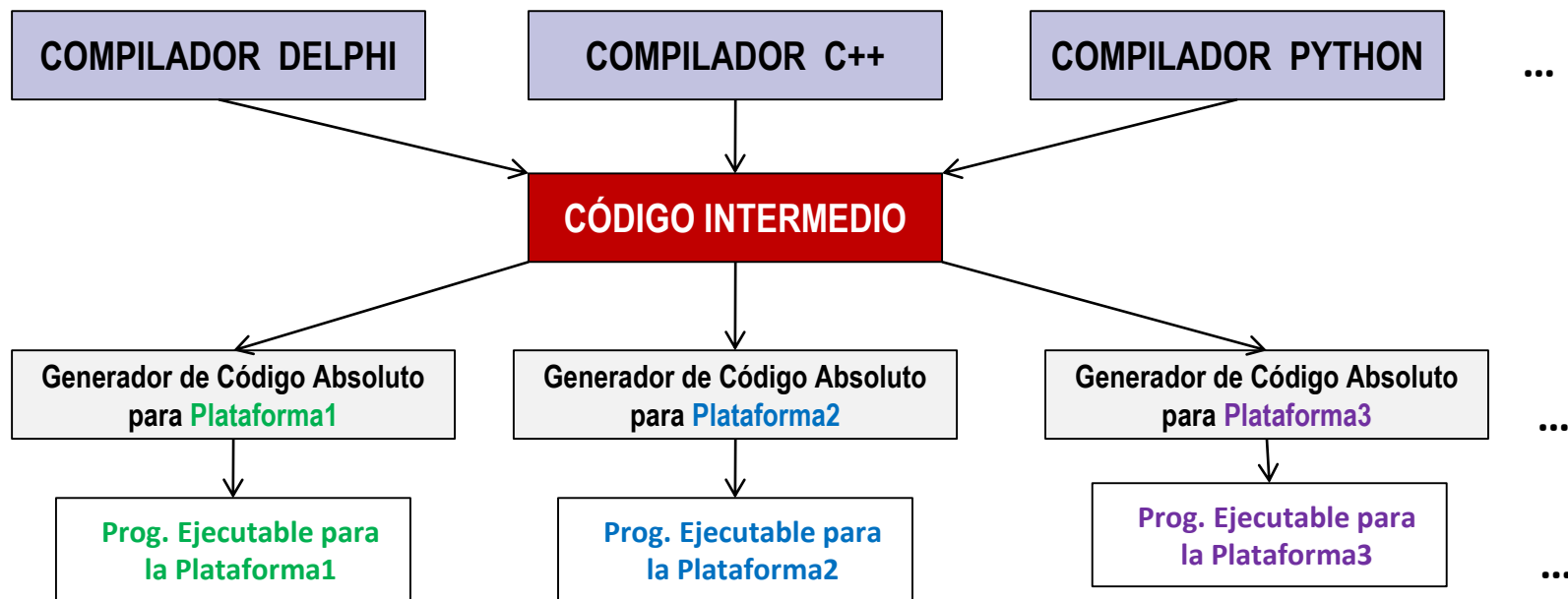
2.1. Front-End y Back-End de un Compilador.

Por ejemplo, si queremos crear un Compilador de DELPHI para **Windows** y para la **IBM AS400** –Sistema Operativo **OS/400** y CPU IBM-PS–, nuestro Compilador (Front-End) generaría sólo **Código Intermedio**, que luego sendos “**Generadores de Código Absoluto**” (Back-End **Compiler’s**), lo convertirán en los Códigos Objetos de los CPU’s involucrados.



2.1. Front-End y Back-End de un Compilador.

Y aún más, podemos tener varios Compiladores (Front-End) que generen el **mismo Código Intermedio** (programa escrito en un **IL**) con sus respectivos Generadores de Código Absoluto (Compiladores Back-End) para distintas **Plataformas***.



*De Wikipedia: “Una **Plataforma** Informática es una combinación de hardware y software utilizada para ejecutar aplicaciones de software”. Entonces **Plataforma** = **Plataforma de Hardware** + **Plataforma de Software**, donde **Plataforma de Hardware** = Arquitectura de la Computadora y **Plataforma de Software** = Sistema Operativo que usa esa Computadora.

Por ejemplo, podemos enumerar algunas **Plataformas** que se nos vienen a la mente:

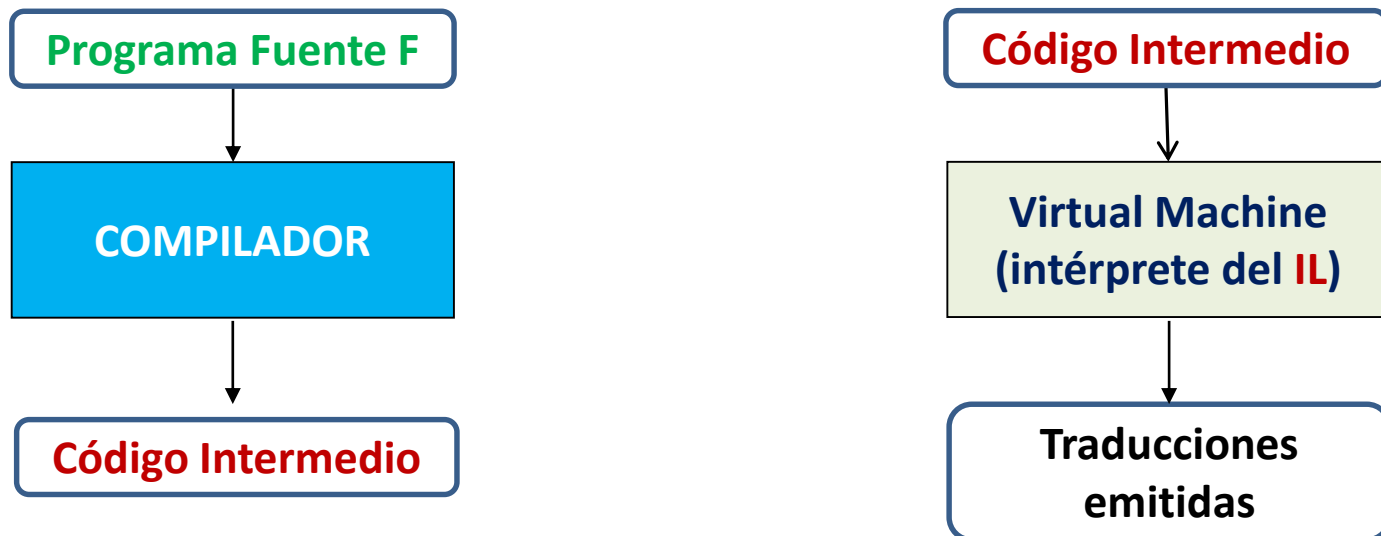
Plataforma1 = IBM-PC INTEL x86 + Windows,
Plataforma3 = Samsung Galaxy S24 + Android,

Plataforma2 = IBM-PC INTELx86 + Linux,
Plataforma4 = iPhone 14 + iOS 17

2.2 Interpretación del IL

También, actualmente, existen compiladores que solo generan el **Código Intermedio**, pero no generan **Código Absoluto** a partir de él.

En vez de eso, emulan la **Virtual Machine (VM)** con un software. Este software en realidad, es un **intérprete** del Lenguaje Intermedio.

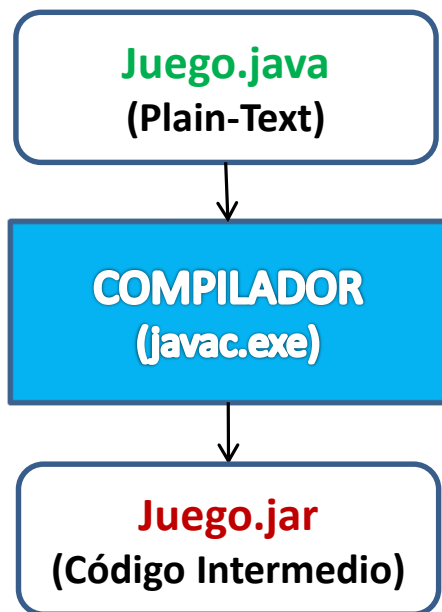


Si quiere rever el concepto de Virtual Machine, haga click [aquí](#).

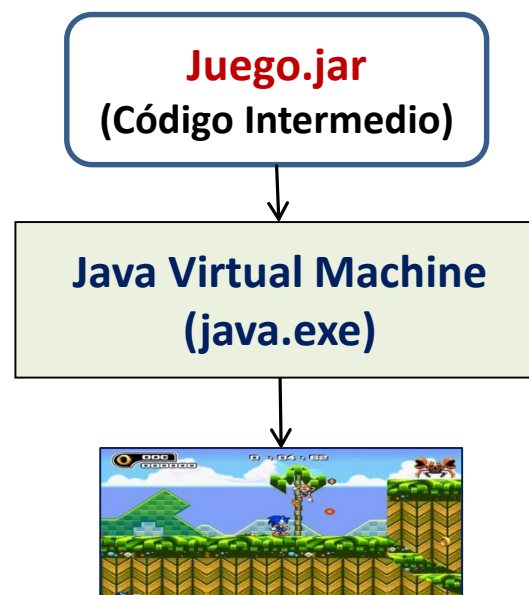
2.2 Interpretación del IL

El lenguaje actual más famoso en esta práctica es JAVA, pues su compilador **javac.exe** traduce el Programa Fuente (**.java**) a un Programa Objeto (**.jar**) escrito en un **IL** diseñado por JAVA.

Compilación en Java



Ejecución en Java



La ejecución (run) del **.jar** la realiza el programa **java.exe**, el cual emula la **Java Virtual Machine** o **JVM**. Es decir, **java.exe**, es un intérprete del **IL** generado por el Compilador.

3. Código de Tres Direcciones

El **Lenguaje Intermedio (IL)** más ampliamente usado es el denominado “**Código de tres Direcciones**” (en inglés: **Three Address Code**), normalmente abreviado 3AC, Código-3 o C3.

Este **IL** se distingue porque cada una de sus instrucciones tiene a lo sumo **tres direcciones de memoria**, y es típicamente una combinación de asignación y operador binario. Por ejemplo, $t1 = t2 + t3$.

Por direcciones de memoria, nos referimos a:

- **Variables** (x, y, t1, t2, ...)
- **Procedimientos/funciones**, y
- **Etiquetas** (E1:, E2:, ...)

3. Código de Tres Direcciones

Por ejemplo:

$x = 10$

Correcto: Usa una dirección (x)

$z = y * w$

Correcto: Usa 3 direcciones (z, y, w)

$q = w$

Correcto: Usa 2 direcciones (q, w)

$p = y/t1 + z$

Incorrecto: Usa 4 direcciones (p, y, t1, z)

$m = m*y + m$

Incorrecto: Usa 4 direcciones.

3.1 Convenciones sobre los ID's del C3

20

Las Etiquetas (Labels) solo pueden ser anotadas como E1:, E2:, E3:, etc. No se permiten nombres de libre albedrío, como se hace en el lenguaje de algún ensamblador.

Todas las Variables se consideran numéricas (en este curso solo usamos variables enteras). Los **nombres** de las **variables** y **procedimientos** de un programa C3, tienen el mismo patrón de formación de las variables de un programa de un lenguaje de alto nivel (Delphi, Java, etc).

Pero, al convertir sentencias de un lenguaje de alto nivel a sentencias C3, el compilador necesita crear sus propias variables, llamadas **Temporales** (auxiliares). Estas variables temporales, se anotan como t1, t2, t3, etc.

3.1 Convenciones sobre los ID's del C3

21

Por ejemplo, la siguiente sentencia en Java

z = x * (y + w) / a;

ocupa 5 direcciones de memoria (z, x, y, w, a). Escrita en código equivalente C3, se generarían 3 sentencias que utilizarían 2 variables temporales (t1 y t2):

t1 = y + w

t2 = x * t1 //t2 = x *(y+w)

z = t2 / a

3.2 El Juego de Instrucciones C3

22

Asignaciones simples

Var = numero //e.g. x=10, t2 = 60

Var1 = Var2 //e.g. y=x, t1 = z

Operaciones aritméticas (+, −, *, /, MOD)

RECUERDE Toda operación **aritmética** o **lógica** se efectúa **entre variables**. Así, es incorrecto escribir

$$x = z - 10$$

pues el 10 es una constante (un número).

La sintaxis de estas operaciones, se define:

Var1 = Var2 **op** Var3 donde **op** $\in \{+, -, *, /, \text{MOD}\}$

Por ejemplo: $x = y * z$, $t1 = z - t2$

3.2 El Juego de Instrucciones C3

23

Asignación con menos unario

Var1 = **-**Var2 //e.g. x= **-**y

Operaciones lógicas (not, or, and)

Var1 = **not** Var2 //e.g. z = not y

Var1 = Var2 **or** Var3 //e.g. z = y or x

Var1 = Var2 **and** Var3 //e.g. z = y and x

Lógicas con operadores relacionales (OPREL): =, ≠, ≤, <, ≥, >

Var1 = (Var2 oprel Var3) //Los paréntesis son opcionales.

Por ejemplo: z = (x ≠ y), t1 = (x < y)

Tratamiento semántico de los Booleanos

Puesto que nuestro Código de Tres Direcciones (C3), solo manipula números enteros, el tratamiento con los booleanos los efectúa así:

- Cuando se lee (**read**) el valor de una variable z :
Si $z=0$ entonces z es **false**.
Si $z \neq 0$ entonces z es **true**
- Cuando se escribe (**write**) en una variable z ($z = \text{Expresión}$)
Si el resultado de la Expresión es **false** entonces $z=0$
Si el resultado de la Expresión es **true** entonces $z=1$

3.2 El Juego de Instrucciones C3

25

Por ejemplo:

$x = -2$

$p = 0$

$z = x \text{ or } p \quad //z = 1$

Explicando:

Aquí estamos consultando (leyendo, **read**) el valor de x . Como $x \neq 0$, se toma $x = \text{true}$

$z = x \text{ or } p$

Consultando (leyendo, **read**) el valor de p . Como $p = 0$, se toma $p = \text{false}$

Aquí estamos almacenando (escribiendo, **write**) en z el valor de la expresión. Como el resultado es **true** ($x \text{ or } p = \text{true or false} = \text{true}$), se escribe **1** en z .

3.2 El Juego de Instrucciones C3

26

Otro ejemplo:

y = 50

t1 = -100

z = **not** y //z = 0 (y=50=true $\Rightarrow$ z = **not** y = **not** true = false = 0)

q = t1 **and** y //q = 1 (t1 = -100=true; y=50=true $\Rightarrow$ q=true **and** true=1)

Otro ejemplo:

z = 50

y = 200

t1 = (z < y) //t1=1 (true), porque z < y (50 < 200)

t2 = (z ≥ y) //t2=0 (false), porque z NO es ≥ y

t3 = (z ≤ y) //t3=1 (true)

m = (z ≠ y) //m=1 (true)

3.2 El Juego de Instrucciones C3

27

Operaciones de Incremento y Decremento.

INC var //Produce $\text{var} = \text{var} + 1$

DEC var //Produce $\text{var} = \text{var} - 1$

Salto Incondicional. Produce un salto a una etiqueta

GOTO etiqueta //e.g. **GOTO** E2

Salto Condicional. Produce un salto a una etiqueta, dependiendo del valor booleano de una variable.

IF (var=1) $\Rightarrow$ **GOTO** etiqueta //Salta si var es true

IF (var=0) $\Rightarrow$ **GOTO** etiqueta //Salta si var es false

3.2 El Juego de Instrucciones C3

Instrucciones de Biblioteca para la Consola

- **READ** (*var*)

Lee el *valor* de la variable *var* desde el teclado. El programa cuando ejecuta esta orden, hace una pausa mostrando el cursor "_". Entonces, el user introduce el valor de la *var* y luego pulsa [↵ Enter]. (El cursor queda al principio de la siguiente línea)

- **WRITE**(*var*)

Muestra en consola el *valor* de *var*, dejando el cursor al lado de lo impreso.

- **WRITES**("mensaje")

Muestra en consola el *mensaje*, dejando el cursor al lado de lo impreso.

3.2 El Juego de Instrucciones C3

- **NL**

NewLine. Hace que el cursor baje al principio de la siguiente línea.

Por ejemplo: El siguiente programa C3

```
writeS("Introduzca")
writeS(" n:")
read(n)
writeS("Lo introducido es ")
write(n)
NL
writeS("bye")
```

se comporta así:

3.2 El Juego de Instrucciones C3

30

```
writeS("Introduzca")
```

```
Introduzca_
```

```
writeS(" n:")
```

```
Introduzca n:_
```

```
read(n)
```

```
//El user ingresa n=123
```

```
Introduzca n: 123 ↵
```

```
_
```

```
writeS("Lo introducido es ")
```

```
Introduzca n: 123  
Lo introducido es _
```

```
write(n)
```

```
Introduzca n: 123  
Lo introducido es 123_
```

```
NL
```

```
//El cursor baja a la sigte línea.
```

```
Introduzca n: 123  
Lo introducido es 123
```

```
_
```

```
writeS("bye")
```

```
Introduzca n: 123  
Lo introducido es 123  
bye_
```

3.2 El Juego de Instrucciones C3

Ejercicios. Convierta las siguientes expresiones JAVA a C3.

(a) $x = y / z;$

Solución. Puesto que la expresión solo usa 3 direcciones de memoria, el resultado es el mismo:

$$x = y / z$$

(b) $p = x * 10;$

Solución. Dado que no se permiten operaciones con constantes, usamos una variable temporal (auxiliar) $t1$, para almacenar el **10**:

$$t1 = 10$$

$$p = x * t1$$

3.2 El Juego de Instrucciones C3

32

(c) $x = y + z \% w - 5;$ $// \% = \text{mod}$

Solución. Esta expresión usa 4 direcciones de memoria y también una ctte (el 5). La calculamos “poco a poco”, usando temporales:

$t1 = z \text{ mod } w$

$t2 = y + t1$

$t3 = 5$

$x = t2 - t3$

$//t2 = y + z \% w$

$//\text{Para realizar la operación } -$

(d) $b = \text{true};$

Solución. Como se mencionó, el C3 anota con 1 al true:

$b = 1$

3.2 El Juego de Instrucciones C3

33

(e) `b = false;`

Solución. Como se mencionó, el C3 anota con 0 al false:

`b = 0`

(f) `sw = (x == y) || (z != 0);`

Solución.

`t1 = (x = y)`

`t2 = 0` //Para realizar la operación $\neq$

`t3 = (z  $\neq$  t2)`

`sw = t1 or t3`

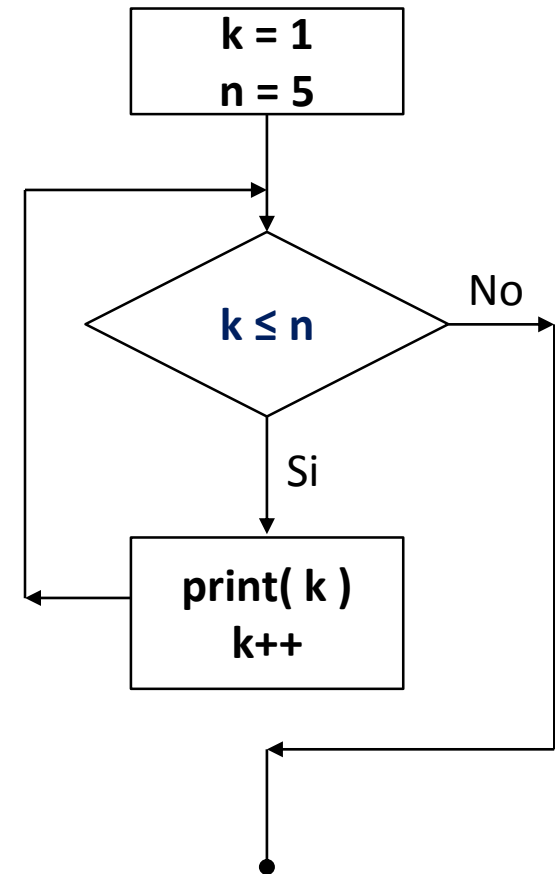
3.2 El Juego de Instrucciones C3

34

Ejercicio. Convierta el siguiente código JAVA a C3.

```
k = 1;  
n = 5;  
while (k <= n){  
    print( k );  
    k++;  
}
```

Solución. Para entender mejor la conversión, vemos un diagrama de flujo del código JAVA.



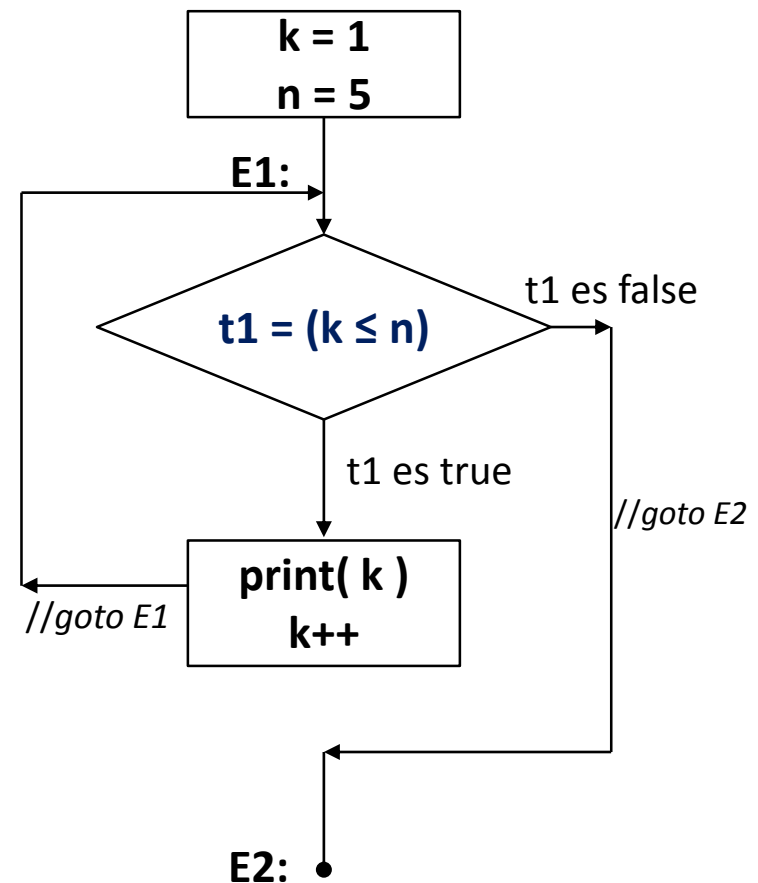
3.2 El Juego de Instrucciones C3

35

Ahora, ponemos etiquetas (E1 y E2) a los saltos y calculamos la expresión booleana del while en una variable temporal t1. El código resultante es:

```
k=1
n=5
E1:
  t1 = (k ≤ n)
  if (t1=0) ⇒ goto E2    //Si t1 es false
  write( k )    //print( k )
  inc k         //k ++
  goto E1
```

E2:



3.2 El Juego de Instrucciones C3

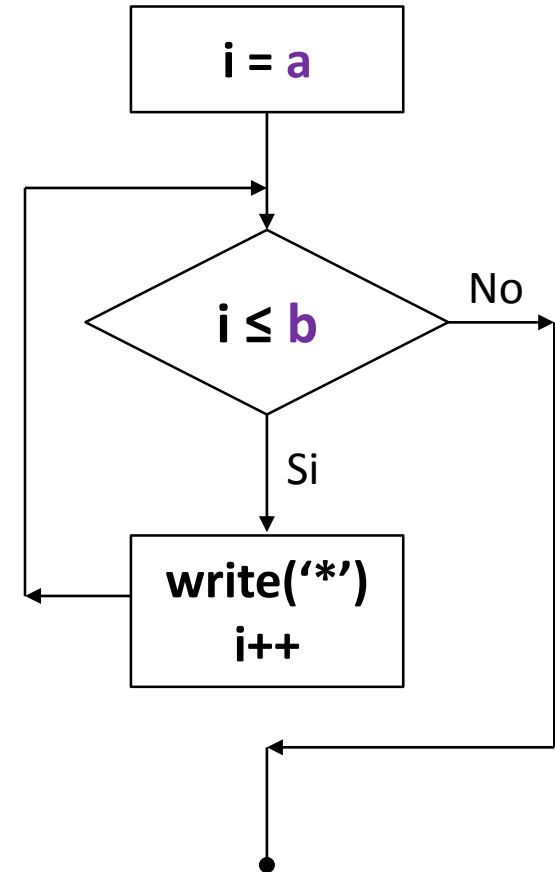
36

Ejercicio. Convierta el siguiente código Pascal a C3

```
for i=a to b do  
begin  
  write('*' );  
end;
```

Solución. Como se sabe, un for, es en realidad un while:

```
i=a ;  
while (i <= b){  
  write('*' );  
  i ++ ;  
}
```

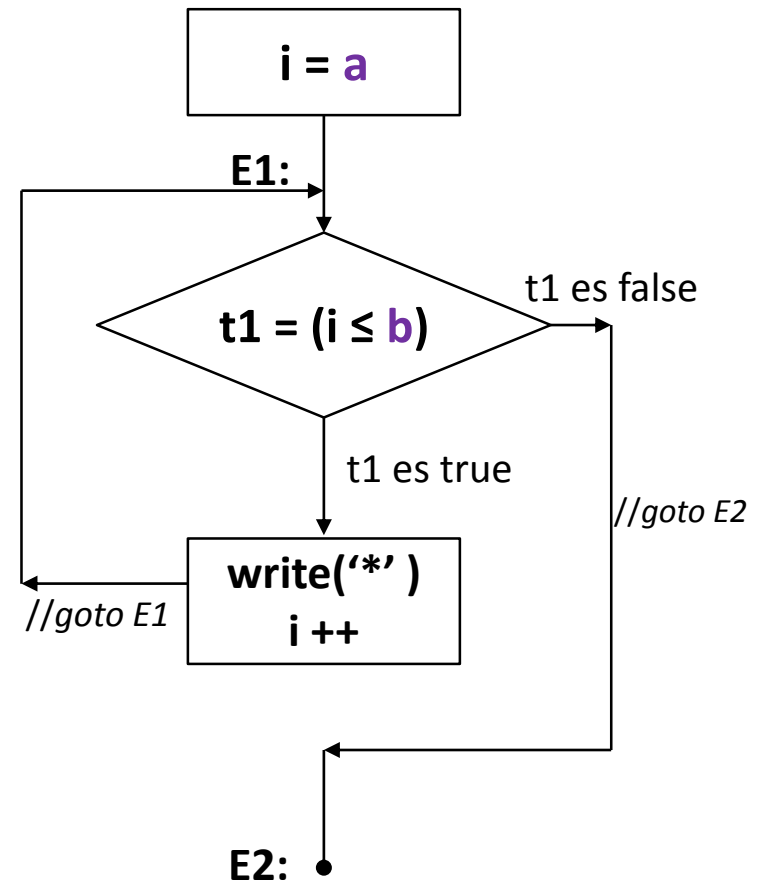


3.2 El Juego de Instrucciones C3

37

Entonces, procedemos como en el ejercicio anterior:

```
i = a  
E1:  
  t1 = (i ≤ b)  
  if (t1=0) ⇒ goto E2    //Si t1 es false  
  writeS("*")  
  inc i          // i++  
  goto E1  
E2:
```

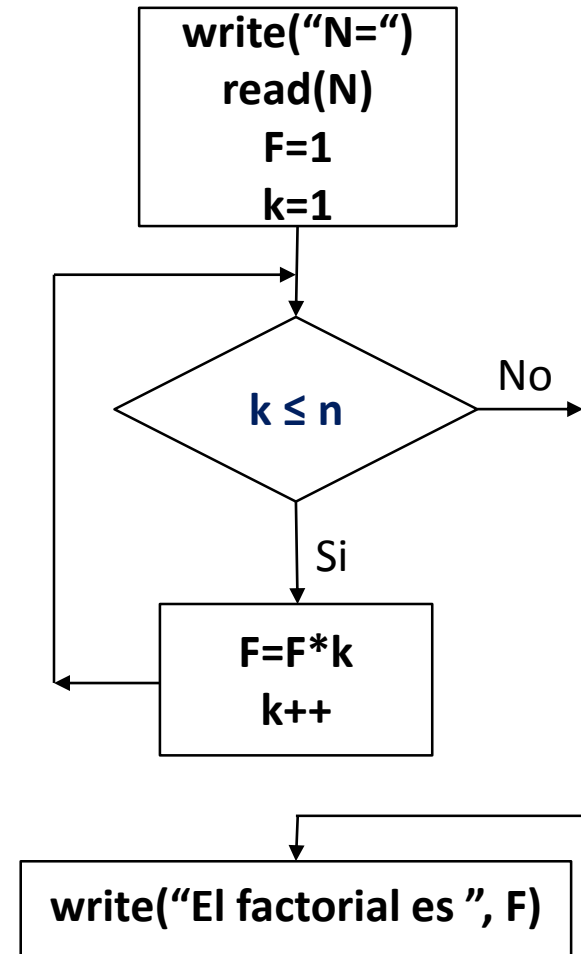


3.2 El Juego de Instrucciones C3

Ejercicio. Escriba un programa C3, que lea un número N y calcule en F el factorial de N ($F=1 \times 2 \times 3 \times \dots \times N$).

Solución. Escribiremos el código en alto nivel y luego lo pasaremos a C3. *Podríamos usar un for, pero luego tendríamos que convertirlo a un while.*

```
write("N=");  
read(N);  
F=1;  
k=1;  
while (k <= N){ //for k=1 to N  
    F = F * k;  
    k ++;  
}  
write("El factorial es ", F);
```



3.2 El Juego de Instrucciones C3

39

El C3 resultante es:

```
writeS("N=")
```

```
read(N)
```

```
F=1
```

```
k=1
```

E1:

```
t1 = ( $k \leq n$ )
```

```
if (t1=0)  $\Rightarrow$  goto E2 //Si t1 es false
```

```
F = F * k
```

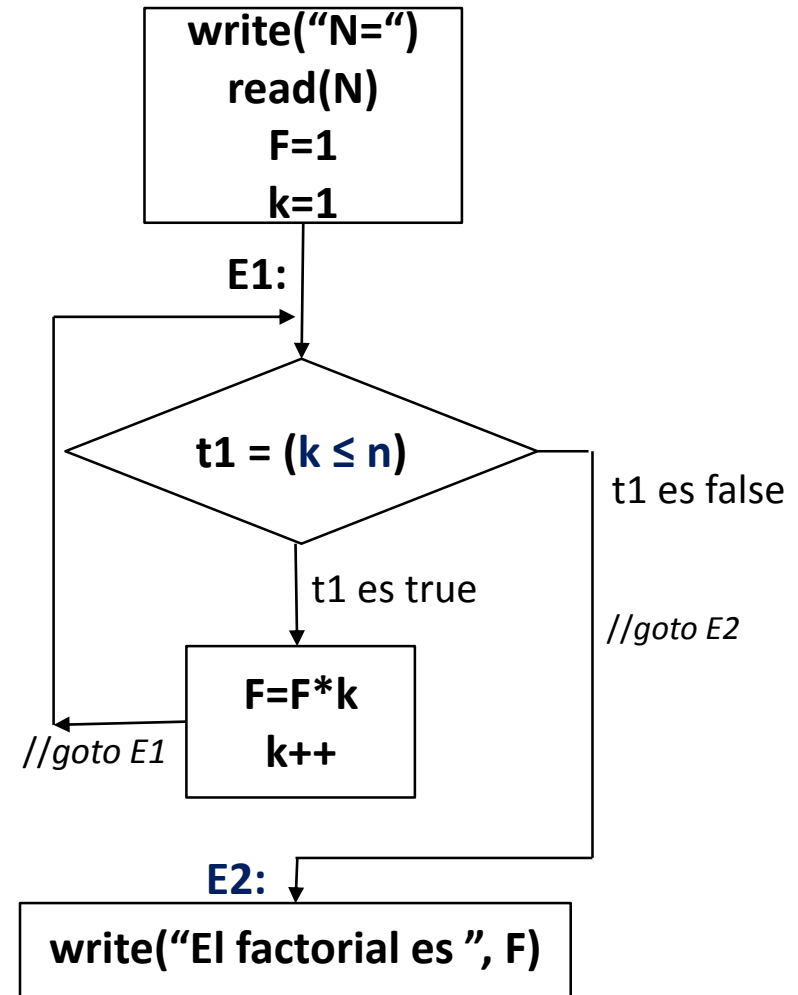
```
inc k //k ++
```

```
goto E1
```

E2:

```
writeS("El factorial es ")
```

```
write(F)
```



3.3 Procedimientos en el C3

Todo programa C3 está organizado en procedimientos, por lo tanto, un programa C3 tiene **al menos un** procedimiento: *el procedimiento principal*. El C3 destina dos sentencias, para la manipulación de procedimientos:

- **CALL nombre_procedimiento** //e.g. CALL Lectura
Hace que el programa ejecute las líneas del procedimiento llamado.
- **RET**
Return. Línea que se escribe dentro de un procedimiento, para salir de él y volver a la siguiente línea de donde fue llamado (CALL).

Los **nombres** de los **procedimientos** de un programa C3, tienen el mismo patrón de formación que los usados en un lenguaje de alto nivel (Delphi, Java, etc), excepto el del *procedimiento principal* al cual se le llama “\$Main”.

Antes de usar Procedimientos, RECUERDE:

- El C3 de este curso **NO** manipula **variables locales**. Todas las variables usadas, son globales.
- Las **etiquetas** son de **ámbito global**. Es decir, por ejemplo, si el Procedimiento1 usa las etiquetas E1 y E2, entonces el Procedimiento2 (si necesita etiquetas) usará E3, E4, En otras palabras, en **todo** el programa C3 una **etiqueta** solo debe aparecer **una sola vez**.
- Sin embargo, **todas** las variables **temporales** (t1, t2, t3, ...) **son locales**. Es decir, por ejemplo, tanto el Procedimiento1 como el Procedimiento2 puede usar las variables t1, t2, t3, ...

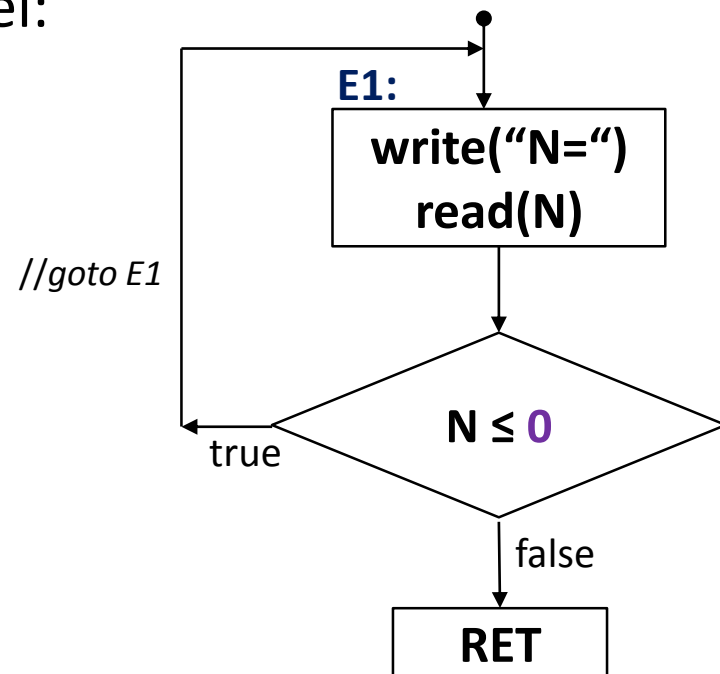
3.3 Procedimientos en el C3

Ejercicio. Lea una variable N y valide que sea mayor que cero. Luego muestre en la consola (pantalla) N asteriscos.

Solución. Podemos utilizar un procedimiento Lectura, para leer y validar N. En el procedimiento principal, llamamos a Lectura y luego mostramos los N asteriscos.

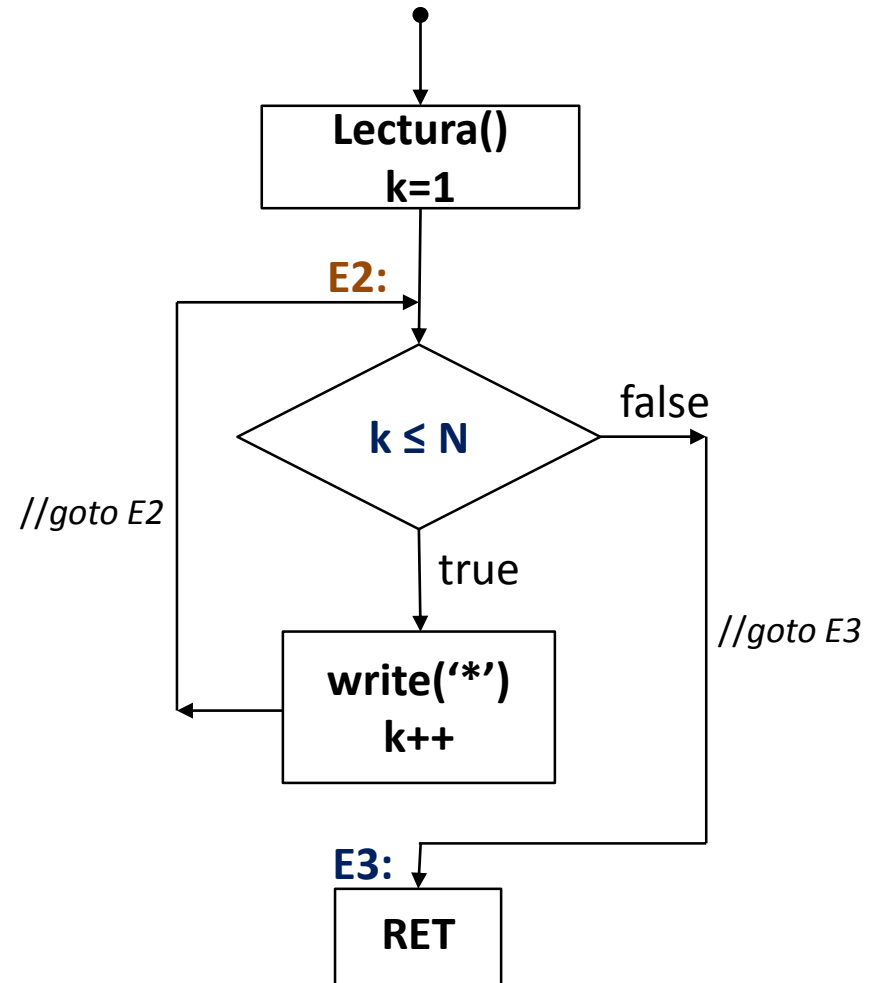
Escribiendo el código en alto nivel:

```
void Lectura() {  
    do{  
        write("N=");  
        read(N);  
    } while (N <= 0);  
} //return
```



3.3 Procedimientos en el C3

```
void $Main() {  
    Lectura(); //Call Lectura  
    k=1;  
    while (k<=N){ //for k=1 to N  
        write("*");  
        k++;  
    }  
} //return
```



3.3 Procedimientos en el C3

El programa C3 queda así:

Proc. Lectura

E1:

```
writeS("N=")
read(N)
t1 = 0
t2 = (N ≤ t1)
if (t2=1) ⇒ goto E1
RET
```

Proc. \$Main

call Lectura //Leer y validar N

k = 1

E2:

```
t1 = (k ≤ N)
if (t1=0) ⇒ goto E3
writeS("*")
inc k // k++
goto E2
```

E3:

RET

4. Esquemas de Traducción C3

Un Esquema de Traducción C3 es un *layout* de código, que define el diseñador del Compilador, para convertir construcciones de programación de un Lenguaje de Alto Nivel a Código de Tres Direcciones (C3).

Por ejemplo. Si se tiene la construcción `while` de JAVA:

```
while (ExprBoole){  
    sentencias;  
}
```

¿Cuál sería la estrategia para convertir ésta construcción a C3?

4.1 Notación Usada en los Esquemas

Para referirse a una sentencia o a un grupo de sentencias convertidas a C3, el diseñador a menudo usa estas abreviaturas.

- ***C3-Sentencia***. Se refiere al grupo de instrucciones C3 que traducen una *Sentencia* de Alto Nivel.

Por ejemplo: A continuación se muestra una *Sentencia* de Alto Nivel (Pascal) y su traducción a C3, es decir, su *C3-Sentencia*.

Sentencia

```
write('El factorial es ', F);
```



C3-Sentencia

```
writeS("El factorial es ")  
write(F)
```

4.1 Notación Usada en los Esquemas

47

- ***C3-Sentencias***. Se refiere al grupo de instrucciones C3 que traducen una serie de *Sentencias* consecutivas de Alto Nivel.

Por ejemplo: A continuación se muestra una serie de *Sentencias* de Alto Nivel (JAVA) y la traducción de ellas a C3, es decir, el *C3-Sentencias* de ésta serie de instrucciones JAVA.

Sentencias

```
K=1;  
Z=K+10;  
println("bye");
```



C3-Sentencias

```
K=1  
t1 = 10  
Z=K+t1  
writeS("bye")  
NL
```

4.1 Notación Usada en los Esquemas

Cuando se traduce una Expresión aritmética (Expr) o una Expresión Booleana (ExprBoole) de Alto Nivel a C3, el resultado es dejado en una variable temporal t_k . Entonces:

- **$C3\text{-Expr}(t_k)$ y $C3\text{-ExprBoole}(t_k)$** . Se refieren a la traducción de una expresión de Alto Nivel a C3, anotando el temporal que contiene el resultado.

Anotamos algebraicamente t_k , en vez de $t1$ o $t2$, ..., porque a priori no se sabe cuantos temporales se usarán para traducir la Expr o la ExprBoole a C3.

Recuerde:

Expr = Expresión aritmética

ExprBoole = Expresión Booleana

4.1 Notación Usada en los Esquemas

Ejemplos: A continuación se muestra la traducción de dos expresiones:

Expr

$z * p + y - 3$

$\Rightarrow$

C3-Expr(t4)

$t1 = z * p$

$t2 = t1 + y$

$t3 = 3$

$t4 = t2 - t3$

El temporal **t4**
tiene el resultado
de la Expr

ExprBoole

$(z \leq p) \text{ or } sw$

$\Rightarrow$

C3-ExprBoole(t2)

$t1 = (z \leq p)$

$t2 = t1 \text{ or } sw$

El temporal **t2**
tiene el resultado
de la ExprBoole

- (i) Escriba un esquema de traducción para la asignación booleana en JAVA:

Var = ExprBoole;

Solución. Simplemente, generamos el C3 de la ExprBoole y el temporal resultado de la misma, se la asignamos a Var:

C3-ExprBoole(t_k)

Var = t_k

(ii) Escriba un esquema de traducción para la asignación aritmética en JAVA:

Var = Expr;

Solución. Procedemos exactamente, como en el ejercicio anterior:

C3-Expr(t_k)

Var = t_k

4.2 Ejercicios

(iii) Usando el Esquema de Traducción anterior, convierta a C3 la siguiente asignación JAVA:

$$\text{Area} = \text{Base} * \text{Altura} / 2;$$

Solución. El Esquema de Traducción anterior, nos dice:

- $C3\text{-Expr}(t_k)$

Primero, convierta la Expr a C3, tomando en cuenta el temporal resultado.

- $\text{Var} = t_k$

Luego, asigne el temporal resultado a la Var.

Entonces, la traducción es:

$$\left. \begin{array}{l} t1 = \text{Base} * \text{Altura} \\ t2 = 2 \\ t3 = t1 / t2 \end{array} \right\} C3\text{-Expr}(t3)$$

$\text{Area} = t3$

4.2 Ejercicios

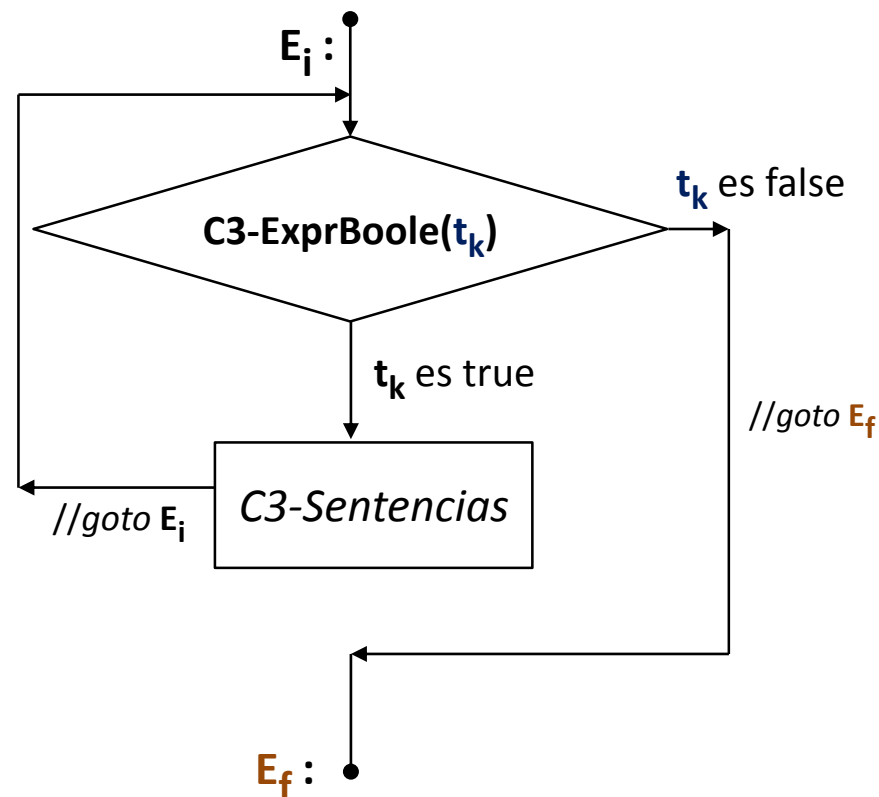
(iv) Escriba un Esquema de Traducción para la construcción while:

```
while (ExprBoole){
    Sentencias;
}
```

Solución.

$E_i :$
 $C3-ExprBoole(t_k)$
 if ($t_k = 0$) $\Rightarrow$ Goto E_f
 $C3-Sentencias$
 Goto E_i

$E_f :$



4.2 Ejercicios

Explicación:

Línea de Código	Explicación
$E_i :$	Incluir un etiqueta al inicio del while. Como no sabemos que número de etiqueta nos toca, escribimos algebraicamente E_i
$C3-ExprBoole(t_k)$	Convertir la ExprBoole del while a C3, cuyo temporal resultado es t_k
if ($t_k = 0$) $\Rightarrow$ Goto E_f	Si la ExprBoole es false, salir del while.
$C3-Sentencias$	Convertir las sentencias del while a C3
Goto E_i	Repetir ciclo
$E_f :$	Incluir un etiqueta para salir del while. Como no sabemos que número de etiqueta nos toca, escribimos algebraicamente E_f

4.2 Ejercicios

55

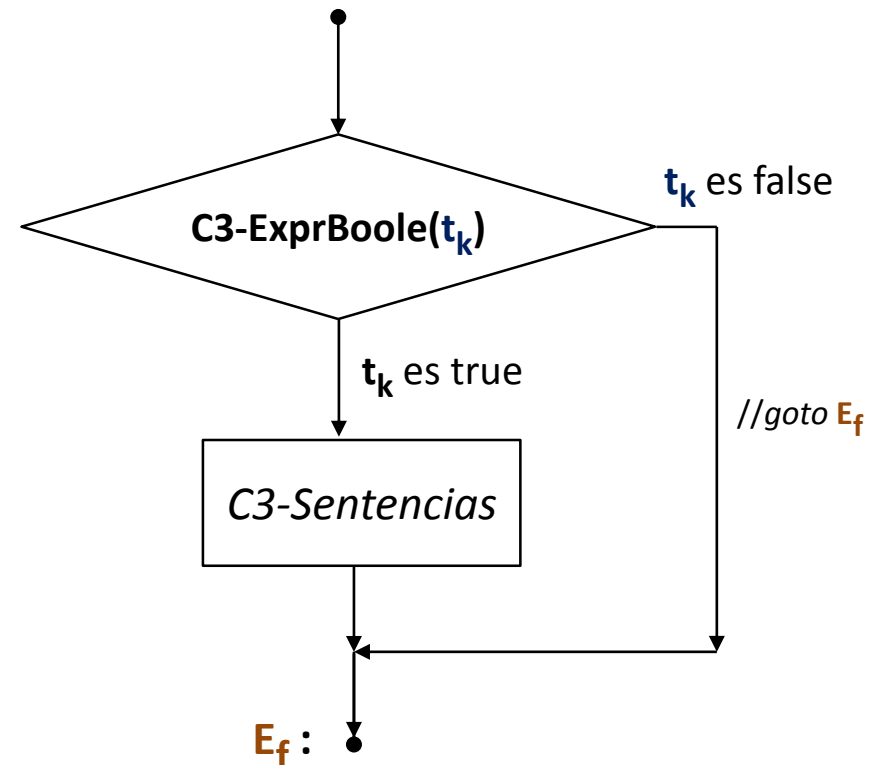
(v) Escriba un Esquema de Traducción para la construcción if:

```
if (ExprBoole){  
    Sentencias;  
}
```

Solución.

$C3\text{-ExprBoole}(t_k)$
if ($t_k = 0$) $\Rightarrow$ Goto E_f
 $C3\text{-Sentencias}$

E_f :



4.2 Ejercicios

(vi) Escriba un Esquema de Traducción del **if-else**:

```

if (ExprBoole){
    Sentencias1;
}
else{
    Sentencias2;
}

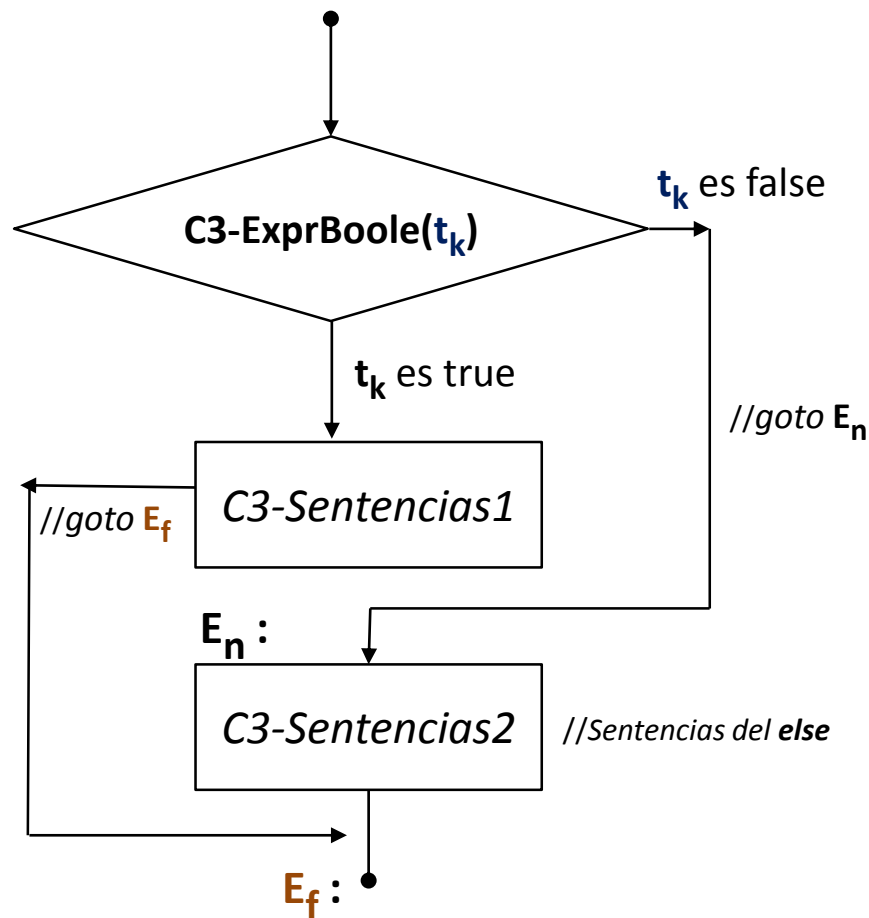
```

Solución.

```

C3-ExprBoole( $t_k$ )
if ( $t_k = 0$ )  $\Rightarrow$  Goto  $E_n$ 
C3-Sentencias1
Goto  $E_f$ 
 $E_n$  :
    C3-Sentencias2
 $E_f$  :

```



4.2 Ejercicios

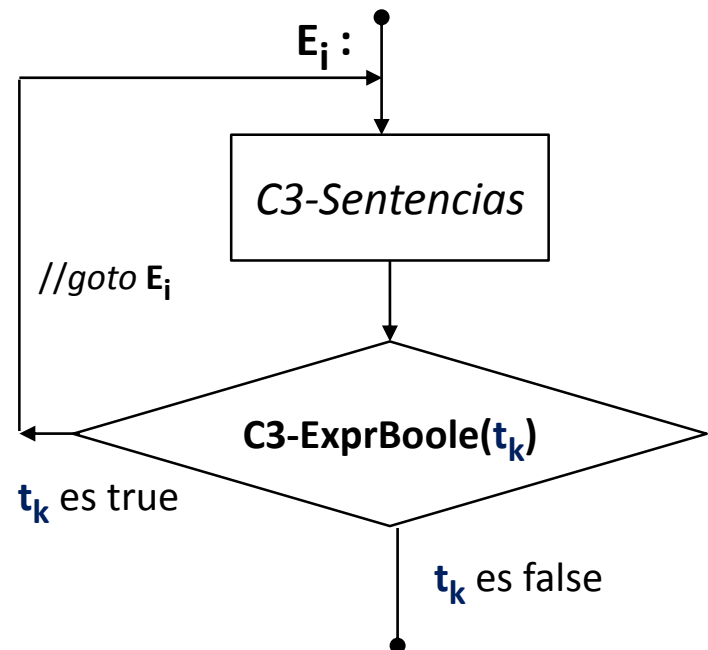
57

(vii) Escriba un Esquema de Traducción para el do-while:

```
do {  
    Sentencias;  
} while (ExprBoole);
```

Solución.

$E_i :$
 C3-Sentencias
 C3-ExprBoole(t_k)
 if ($t_k = 1$) $\Rightarrow$ Goto E_i



5. Representación del C3

Aunque un C3 se escribe textualmente (en modo texto), en realidad, **cada instrucción** de éste código se almacena en una pieza encapsulada de software. La *cantidad de campos* que tiene ésta pieza de software, da nombre a la representación: Si tiene 3, campos se le dice *Terceto*; si tiene 4 campos, *Cuádrupla* o cuádruplo; si tiene 5, *Quíntuplo*.

Tercetos. Usa poca memoria, pero los algoritmos que los tratan son un poco más complicados.

Quíntuplas. Demasiado uso de memoria, algoritmos simples.

Cuádruplas. Serán las usadas en este curso introductorio: usa menos memoria que las Quíntuplas, pero más que los Tercetos; sus Algoritmos son de poca complejidad.

5. Representación del C3

59

Al igual que los Tercetos y las Quintuplas, todos los campos de una **Cuádrupla** (4 campos) son **numéricas**.

TYPE *//En Delphi*

Cuádrupla = RECORD

`opCode` : Integer;

 Dir1, Dir2, Dir3: Integer;

END;

//En JAVA

class Cuádrupla {

public int `opCode`;

public int Dir1, Dir2, Dir3;

}

Pero, independiente del lenguaje que se use, una Cuádrupla puede representarse gráficamente como:



5. Representación del C3

El **opCode** (*Operation Code*) es un **número** que identifica en forma única la operación que harán las tres direcciones siguientes, o sea Dir1, Dir2 y Dir3. Por ejemplo, si tenemos la sentencia **x = y + z**, en la cuádrupla se anota el **opCode** y a continuación las direcciones, en el orden en que aparecen.

opCode	Dir1	Dir2	Dir3
OPSUMA	Dir de x	Dir de y	Dir de z

Donde **OPSUMA** es una constante numérica definida por el programador (e.g. *En Pascal* `const OPSUMA=3;` o *en Java* `public static final int OPSUMA=3`)

5. Representación del C3

Algunas operaciones, como se ha podido notar, no utilizan las tres direcciones. En este caso, a los campos que no usados se los señala con “—”, indicando así, que el valor de ese campo no tiene importancia.

Por ejemplo, la operación **p=not q**, utiliza solamente dos direcciones de memoria:

opCode	Dir1	Dir2	Dir3
OPNOT	Dir de p	Dir de q	—

5. Representación del C3

Entonces, un programa C3 es una *collection* (Vector, Lista, ...) de Cuádruplas. Por ejemplo:

Modo Textual (*irreal*)

call Lect

k = **1**

E1:

t1 = (k ≤ N)

if (t1=0) ⇒ goto **E2**

writeS("*")

inc k

goto **E1**

E2:

RET

Como un Vector de Cuádruplas (*real*)

	opCode	Dir1	Dir2	Dir3
0	OPCall	Dir de Lect	—	—
1	OPNUM	Dir de k	1	—
2	ETIQUETA	1	—	—
3	OPMEI	Dir de t1	Dir de K	Dir de N
4	OPIF0	Dir de t1	2	—
5	OPWRITES	Dir de "*"	—	—
6	OPINC	Dir de k	—	—
7	OPGOTO	1	—	—
8	ETIQUETA	2	—	—
9	OPRET	—	—	—